

Face Entry Pro

Biometric Attendance & Campus Intelligence System

System Design:

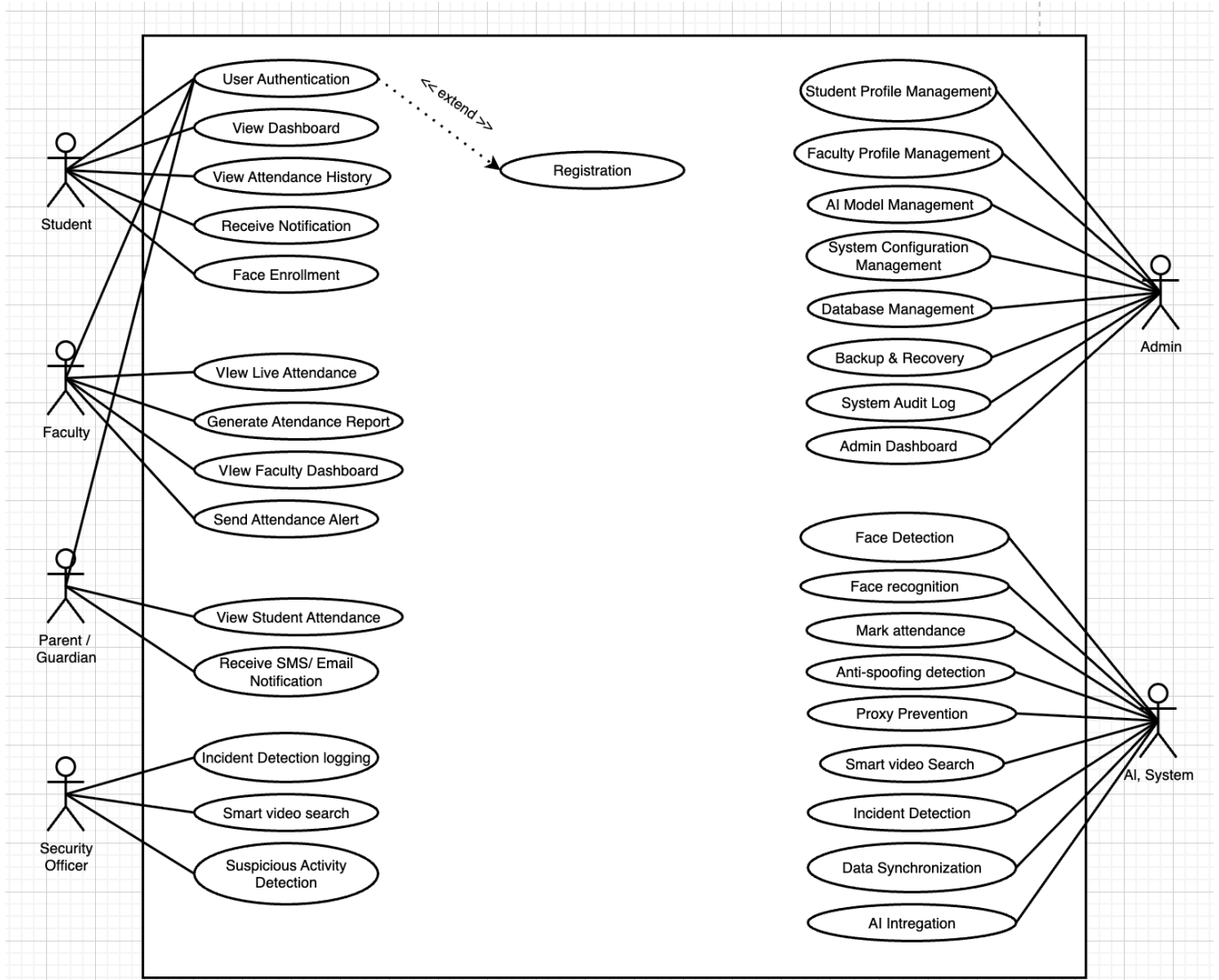
- 1. Use Case Diagram**
- 2. Activity Diagram**
- 3. ER Diagram**
- 4. Sequence Diagram**
- 5. UML Class Diagram**
- 6. Data Flow Diagram**

1. Use case Diagram

The Use Case Diagram provides an overall view of how different users interact with the FaceEntryPro system. It identifies the main actors and the functionalities available to each actor based on their roles and access permissions.

- Using draw.io

Use Case Diagram for Face Entry Pro



Actors: Student, Faculty, Parents/Guardian, Security Officer, Admin, AI, System.

Description

1.1 The System Users

Every successful attendance system depends on the people who use it. In the **FaceEntryPro** system, the main users are:

The Student: Students log in to the system, view their dashboard, check attendance history, receive notifications, and submit feedback. Their attendance is marked automatically through face recognition.

The Faculty: Faculty members monitor real-time attendance, generate attendance reports, analyze attendance records, identify students with low attendance, and send attendance alerts.

The Parent/Guardian: Parents can log in to view their child's attendance records and receive SMS or email notifications about attendance and important updates.

The Security Officer: Security officers monitor campus security by viewing security alerts, searching CCTV footage using face recognition, detecting suspicious activities, and reviewing incident logs.

The Administrator (Admin): Administrators register users, manage student and faculty profiles, enroll facial data, manage cameras, support multiple cameras, store biometric data securely, manage databases, maintain audit logs, perform backup and recovery, and configure the entire system.

The Developer: Developers manage AI models, integrate new AI features, monitor system performance, and ensure the system works properly.

1.2 The AI-Powered Attendance System

FaceEntryPro uses Artificial Intelligence to make attendance faster, more secure, and more reliable.

The Face Recognition System: The system detects faces, recognizes registered users, verifies liveness, prevents proxy attendance through anti-spoofing technology, and marks attendance automatically.

The Attendance Processing System: The system supports offline attendance when there is no internet connection and automatically synchronizes attendance data with the central database when the connection is restored.

The Security Intelligence System: AI searches CCTV footage, detects security incidents, identifies suspicious activities, and helps security officers investigate events quickly.

The Notification Service: The system automatically sends SMS or email notifications to students, parents, and faculty members whenever attendance is low or important updates are available.

The Data Protection System: Biometric information is stored securely using data encryption to protect user privacy and maintain data security.

1.3 The Smart Campus Management

FaceEntryPro combines attendance management, artificial intelligence, and campus security into one smart platform.

The Student Service Center: Students can view attendance records, notifications, dashboards, and submit feedback anytime.

The Faculty Management Center: Faculty members can monitor live attendance, generate reports, analyze attendance trends, and identify students who need attention.

The Parent Monitoring Center: Parents can monitor their child's attendance and stay informed through automatic notifications.

The Campus Security Center: Security officers can search CCTV videos, monitor campus activities, detect suspicious behavior, and review incident records.

The Administration Center: Administrators manage users, biometric enrollment, student and faculty information, cameras, multiple camera locations, databases, audit logs, backups, system configuration, and overall system maintenance.

The AI Management Center: Developers maintain AI models, improve face recognition performance, integrate new AI technologies, and monitor the overall performance of the system to ensure accurate and efficient operation.

2. Activity Diagram

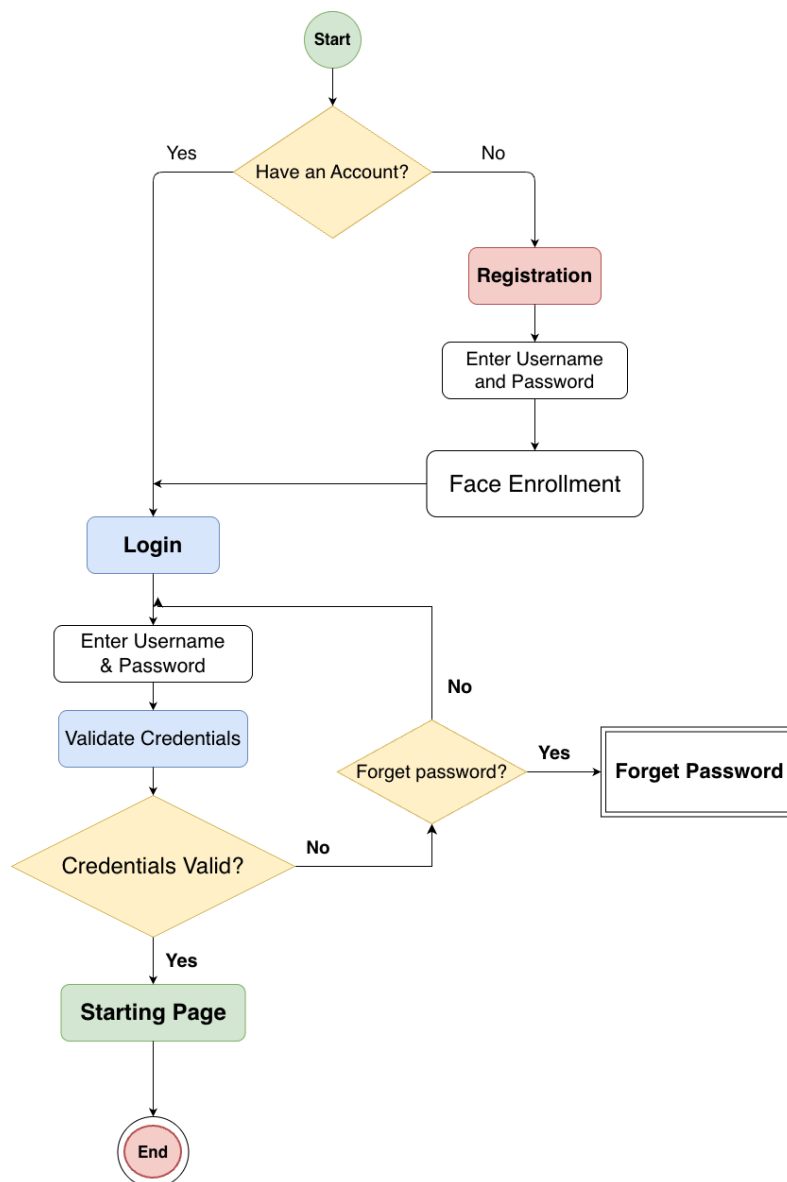
The Activity Diagram illustrates the workflow and sequence of activities performed within the FaceEntryPro system. It shows how a process starts, how different activities are performed, and how the system responds to different conditions and decisions.

The diagram represents the flow from user interaction and authentication to system processing and completion of the required operation. It also shows decision points where the system may follow different paths based on conditions, such as successful or failed authentication and face verification.

Overall, the Activity Diagram provides a clear visualization of the dynamic behavior and operational workflow of FaceEntryPro, helping to understand how different activities are connected and how the system handles various scenarios.

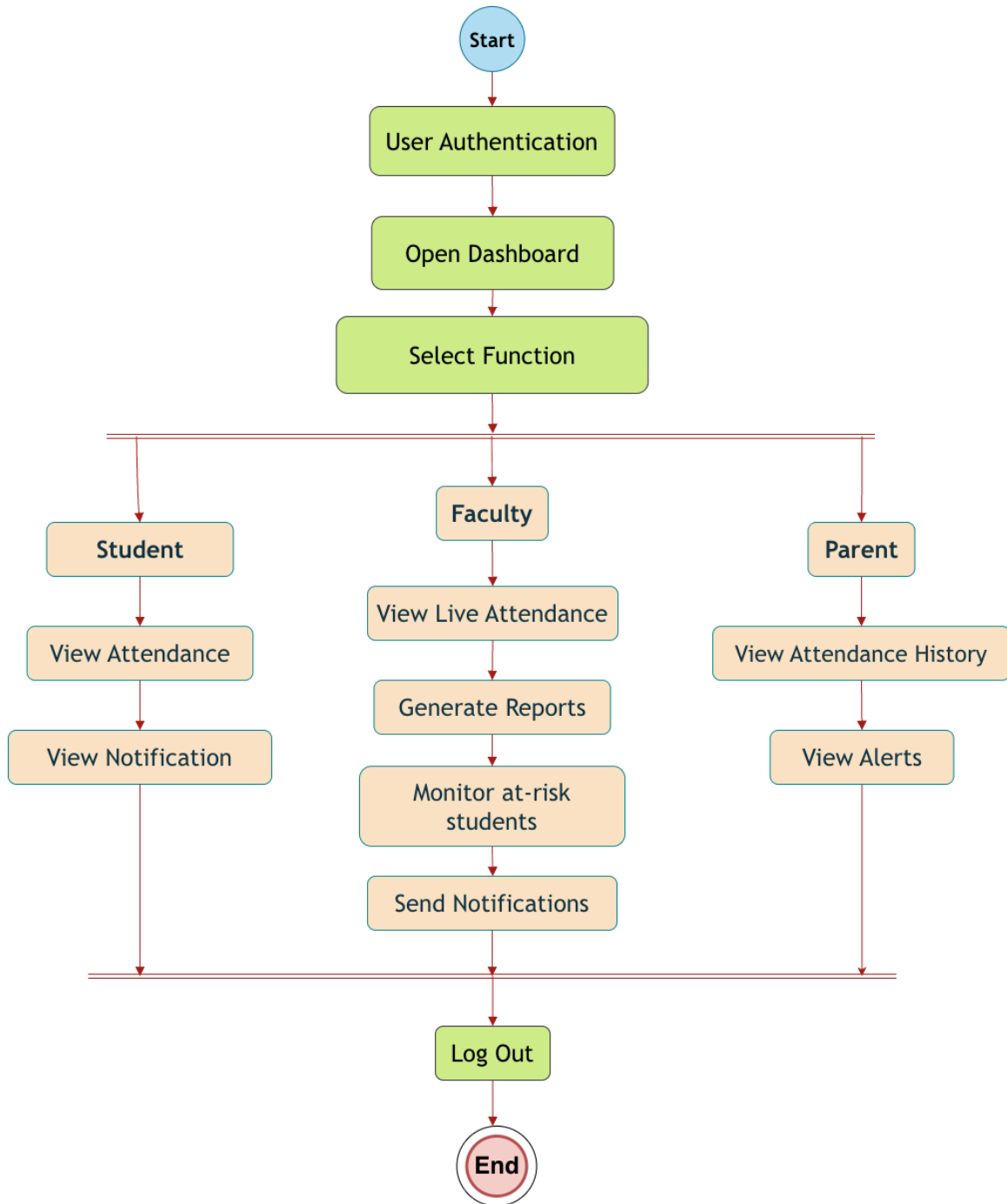
2.1 Authentication Activity Diagram (Using draw.io)

Only for Student :



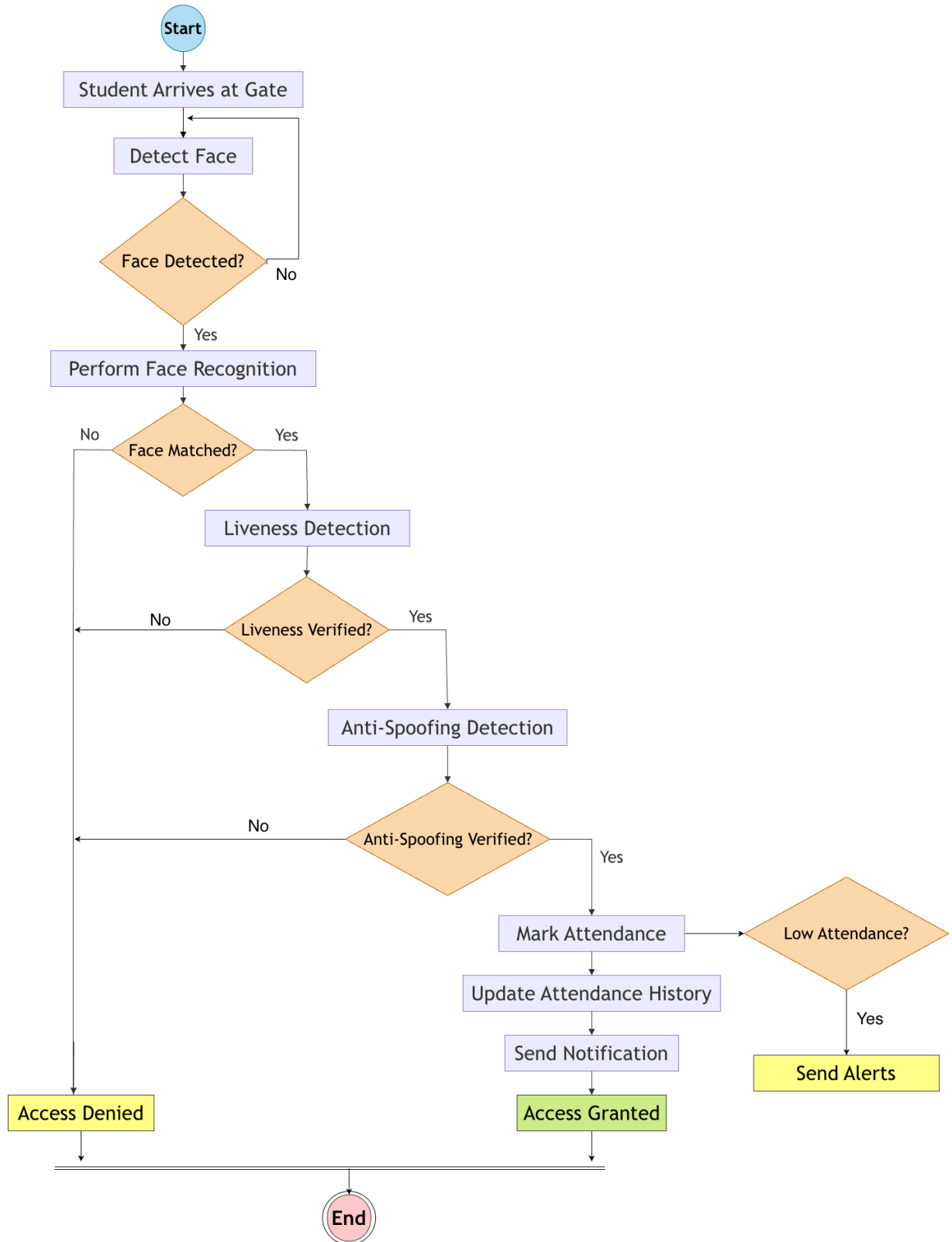
2.2 Overall System Diagram (Using draw.io)

For Student, Faculty, and Parent :



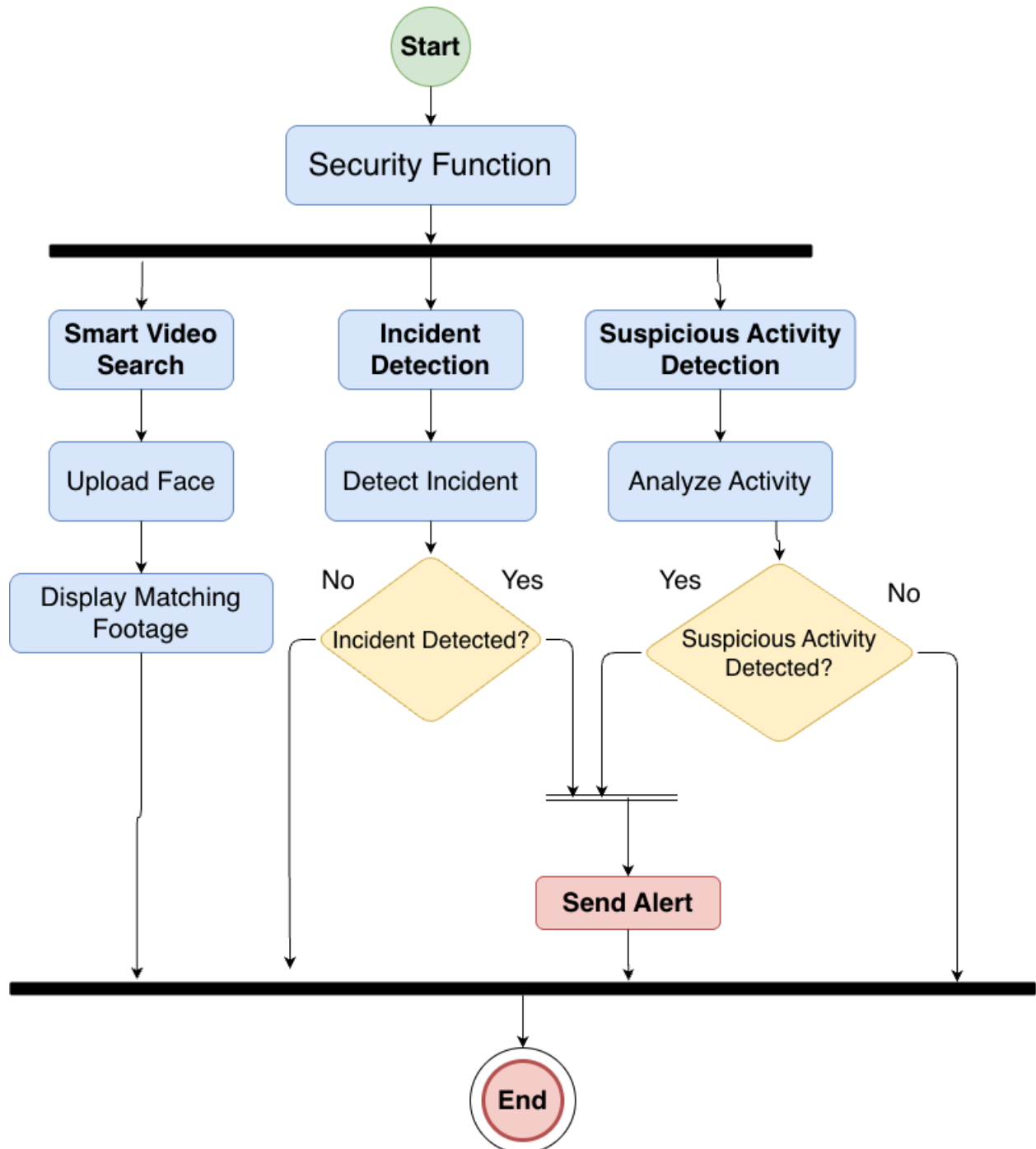
2.3 Campus Intelligence Activity Diagram (Using draw.io)

Shows how the main goal is achieved :



2.4 Campus Security Activity Diagram (Using draw.io)

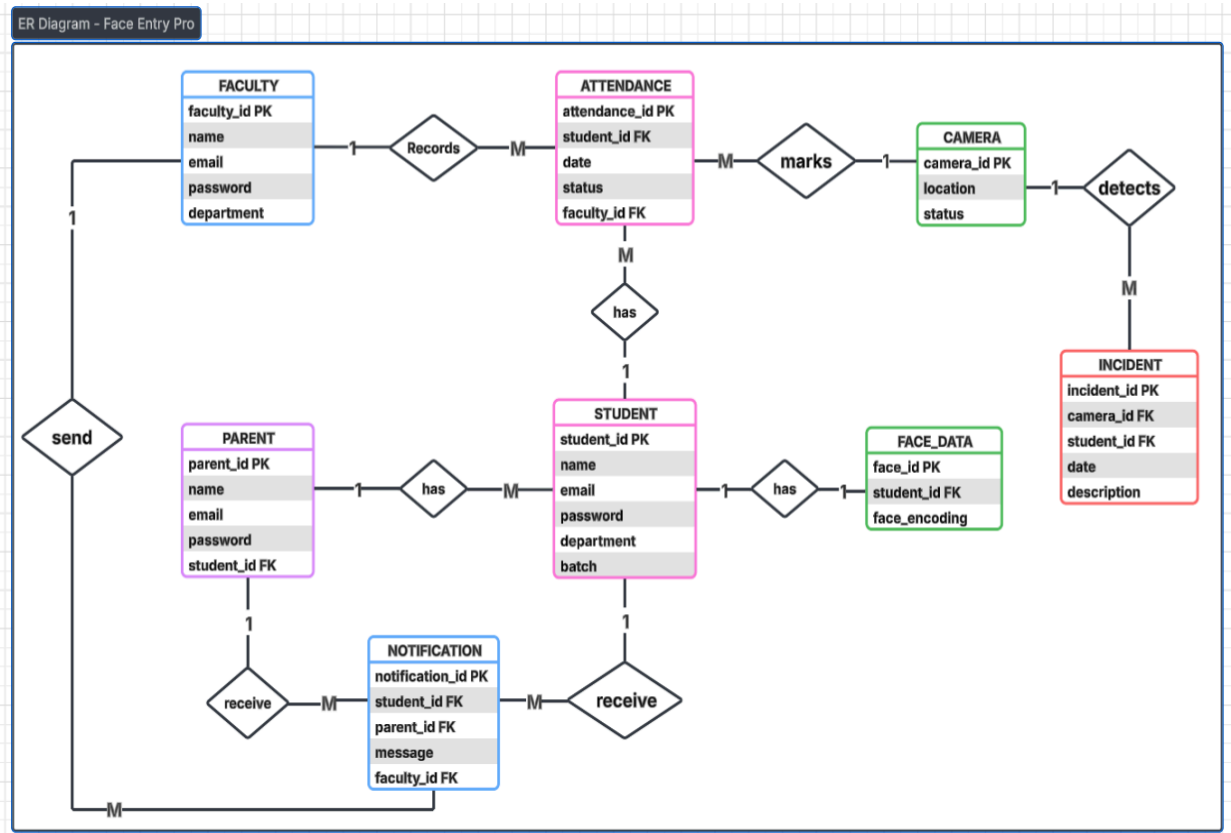
Shows how the security system works :



3. ER Diagram

The Entity-Relationship (ER) Diagram represents the logical structure of the FaceEntryPro database and illustrates how the major entities are connected with each other. It identifies the key entities, their attributes, and the relationships required to store and manage system information efficiently.

ER Diagram (Using Lucid Chart)



ER Diagram Public Link (Lucid Chart):

https://lucid.app/lucidchart/ee6c97eb-25ea-44b9-999a-3a6e84a02cf1/edit?viewport_loc=-1094%2C-2438%2C3454%2C1788%2C0_0&invitationId=inv_c6780665-4255-4aa7-91c1-1a7d4a05046e

3.1 Entity and Attributes

1. Student

Description: Stores the information of registered students.

Attributes:

- Student_ID (PK)
- Name
- Email
- Password
- Department
- Batch

2. Faculty

Description: Stores faculty information responsible for managing attendance.

Attributes:

- Faculty_ID (PK)
- Name
- Email
- Password
- Department

3. Parent

Description: Stores parent or guardian information linked to students.

Attributes:

- Parent_ID (PK)
- Name
- Email
- Password
- Student_ID (FK)

4. Attendance

Description: Stores attendance records of students.

Attributes:

- Attendance_ID (PK)
- Student_ID (FK)
- Faculty_ID (FK)
- Date
- Status

5. Face_Data

Description: Stores facial biometric data for student identification.

Attributes:

- Face_ID (PK)
- Student_ID (FK)
- Face_Encoding

6. Camera

Description: Stores camera information used for face detection and monitoring.

Attributes:

- Camera_ID (PK)
- Location
- Status

7. Incident

Description: Stores security incidents detected by the surveillance system.

Attributes:

- Incident_ID (PK)
- Camera_ID (FK)
- Student_ID (FK)
- Date
- Description

8. Notification

Description: Stores attendance alerts and messages sent to students and parents.

Attributes:

- Notification_ID (PK)
- Student_ID (FK)

- Parent_ID (FK)
- Faculty_ID (FK)
- Message

3.2 Relationship

Relationship	Cardinality	Description
Student – Face_Data	1 : 1	Each student has one facial data record used for biometric recognition.
Student – Attendance	1 : M	A student can have multiple attendance records throughout the semester.
Faculty – Attendance	1 : M	A faculty member can record attendance for multiple students.
Parent – Student	1 : M	A parent can be associated with one or more students.
Student – Notification	1 : M	A student can receive multiple attendance-related notifications.
Parent – Notification	1 : M	A parent can receive multiple attendance notifications regarding their child.
Faculty – Notification	1 : M	One faculty member can send multiple notifications to students and parents.
Camera - Attendance	1 : M	One camera can capture and mark attendance for multiple students.
Camera – Incident	1 : M	One camera can detect multiple security incidents, while each incident is detected by one camera.

4. Sequence Diagram

The Sequence Diagram is a behavioral UML diagram used to represent the dynamic interactions between actors and system components during the execution of a specific process. Unlike static diagrams, it focuses on the chronological order in which messages are exchanged, allowing developers and stakeholders to understand how different objects collaborate to accomplish a particular task.

In the FaceEntryPro system, the Sequence Diagrams illustrate the communication flow for the major functionalities, including user authentication, face recognition-based attendance, and notification management. Each diagram identifies the participating actors, system modules, and database interactions, while clearly demonstrating the sequence of requests, responses, validations, and processing steps required to complete an operation.

These diagrams play an important role in system analysis and design by providing a detailed visualization of the system's runtime behavior. They help developers understand component interactions, verify the correctness of system logic, identify communication dependencies, and ensure that functional requirements are accurately implemented. Additionally, Sequence Diagrams serve as a valuable reference during system development, testing, maintenance, and future enhancements by documenting the interaction patterns between different modules of the application.

Objectives of the Sequence Diagram

- To illustrate the interaction between actors and system components.
- To represent the chronological sequence of messages exchanged during system operations.
- To visualize the runtime behavior of the system.
- To identify the responsibilities of each system component.
- To validate the workflow of functional requirements before implementation.
- To support system design, testing, debugging, and future maintenance.

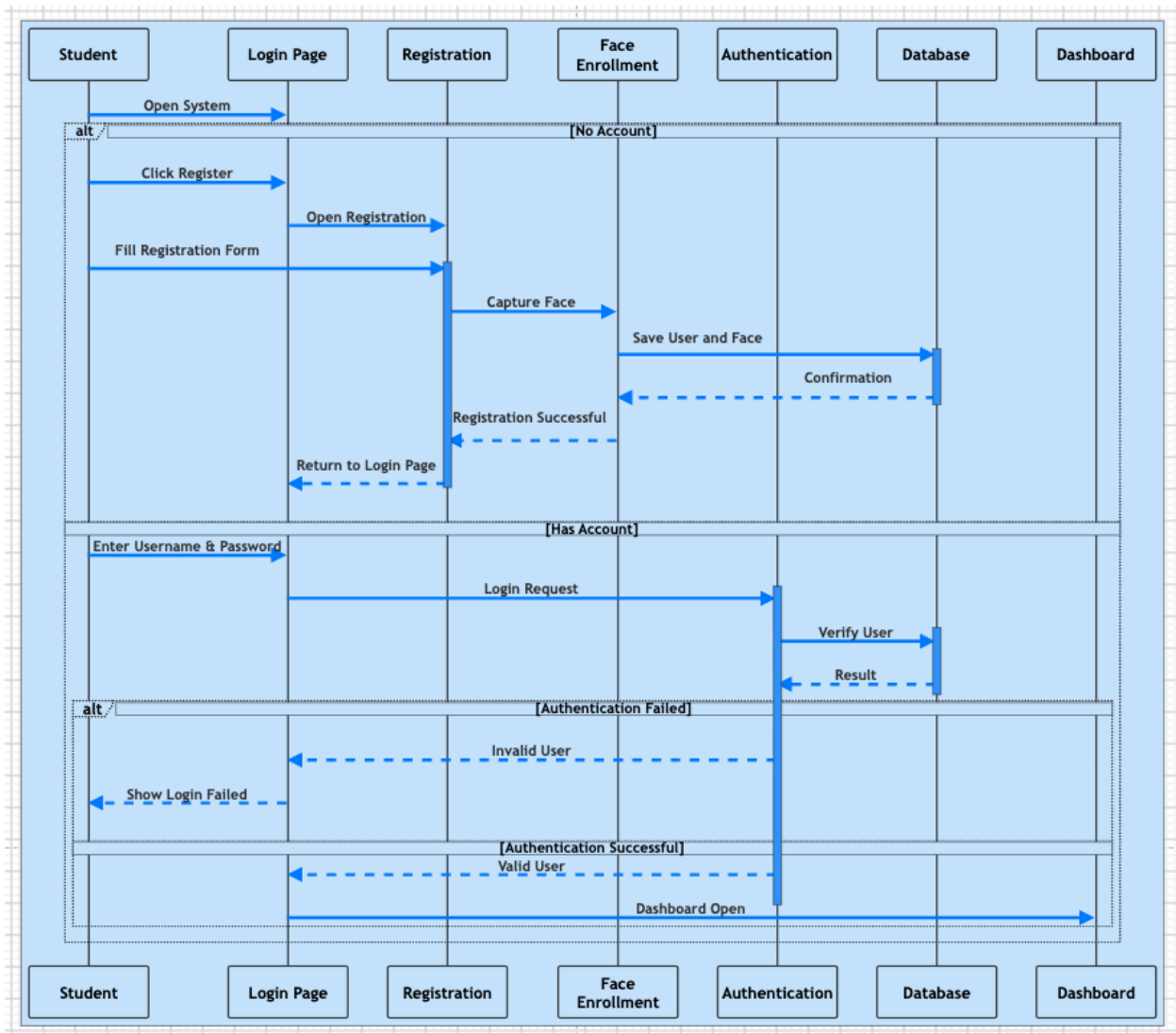
Components of a Sequence Diagram

- Actor: Represents the external user or entity that initiates the interaction.
- Lifeline: Represents the system components or modules involved in the process.
- Message: Indicates the communication or request exchanged between objects.

- Activation: Represents the period during which an object performs an operation.
- Alternative (alt): Represents conditional execution paths, such as success or failure scenarios.
- Return Message: Represents the response returned after processing a request.

4.1 Authentication Sequence Diagram

- Using draw.io



Participants involved: Student → Login Page → Registration → Face Enrollment → Authentication → Database → Dashboard

Steps in the process:

1. **Open System** – The student opens the system and reaches the Login Page.
2. The process then splits into two paths, depending on whether the student already has an account: **No Account** or **Has Account**.

Path 1: No Account (New User Registration)

3. **Click Register** – The student clicks the register option to create a new account.
4. **Open Registration** – The Login Page opens the Registration page.
5. **Fill Registration Form** – The student fills in their personal details.
6. **Capture Face** – The Registration page asks the Face Enrollment module to capture the student's face.
7. **Save User and Face** – The Face Enrollment module sends the student's details and face data to the Database to be saved.
8. **Confirmation** – The Database confirms that the data has been saved successfully.
9. **Registration Successful** – The Registration page informs the student that registration is complete.
10. **Return to Login Page** – The student is sent back to the Login Page to log in with their new account.

Path 2: Has Account (Existing User Login)

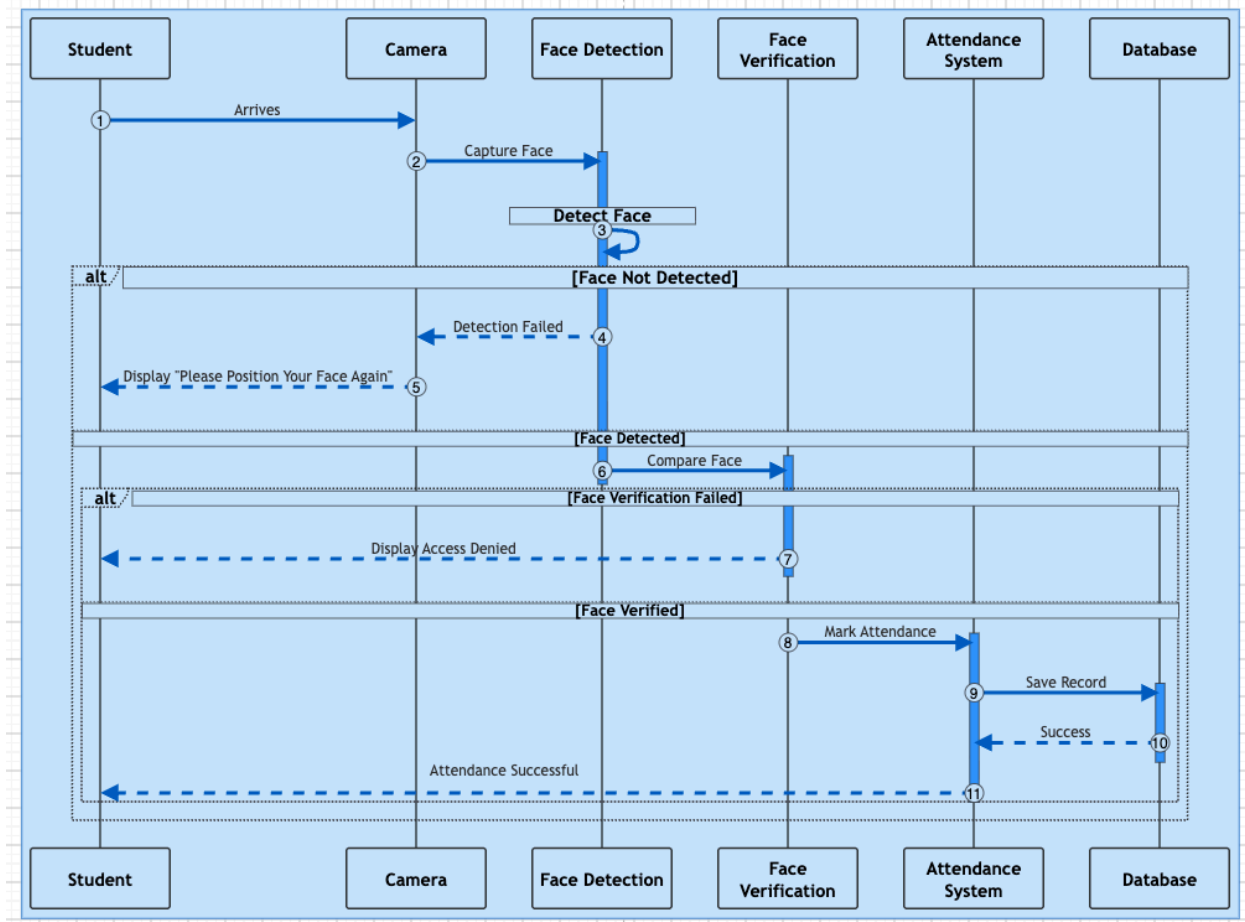
11. **Enter Username & Password** – The student enters their login details.
12. **Login Request** – The Login Page sends this request to the Authentication module.
13. **Verify User** – The Authentication module checks these details against the Database.
14. **Result** – The Database sends back the result of this check.

The outcome then branches into two further cases:

- **Authentication Failed** – If the details are incorrect, the Database sends an "Invalid User" message, and the student sees a "Show Login Failed" message.
- **Authentication Successful** – If the details are correct, the Database confirms the user is valid, and the system opens the **Dashboard** for the student.

4.2 Face Recognition and Mark Attendance Sequence Diagram

- Using draw.io



Participants involved: Student → Camera → Face Detection → Face Verification → Attendance System → Database

What happens, step by step:

1. **Compare Face** – The Face Detection part sends the face it found to the Face Verification part, to check if it matches any student in the system.
2. **If the face does NOT match (Face Verification Failed)** → The system shows "**Access Denied**" to the student.
3. **If the face DOES match (Face Verified)** → The system tells the Attendance System to mark the student as present.

4. **Save Record** – The Attendance System sends this attendance data to the Database to save it.
5. **Success** – The Database tells the Attendance System that the record was saved properly.

Description

This part of the system checks "**Is this really a registered student?**"

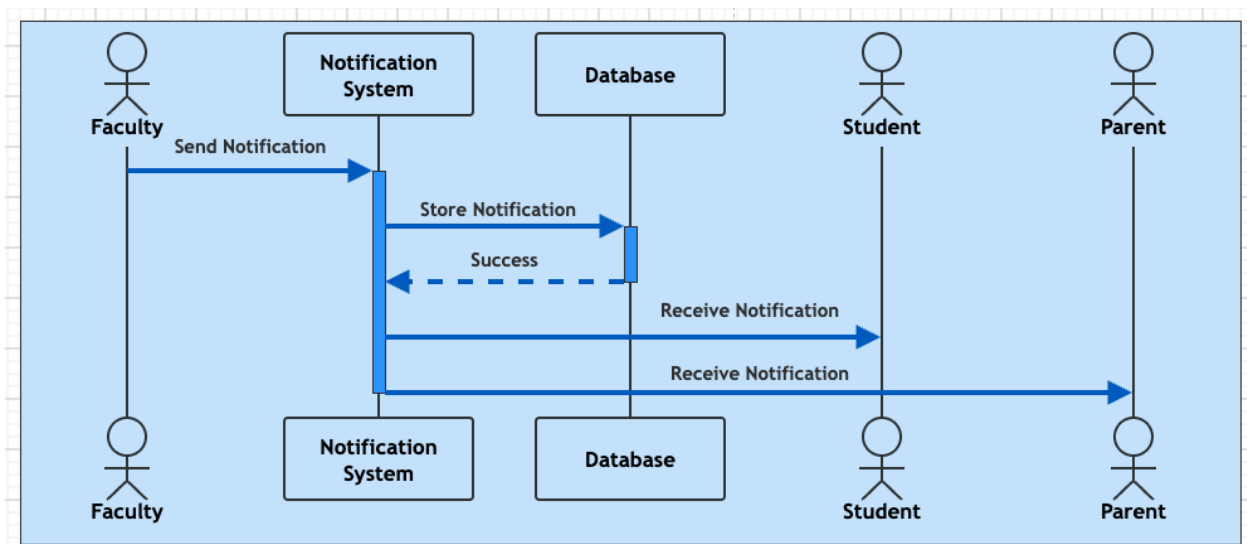
After the camera detects a face, that face is sent to the Face Verification step. This step compares the face with the faces already saved in the system.

- If the face **doesn't match** anyone → the system stops and shows "**Access Denied.**"
- If the face **matches** a student → the system marks that student **present** and saves this information in the Database.

In short, this is the most important step of the whole system, because it decides **whether a real, known student gets marked present or not**. It stops fake or unknown faces from being added as attendance.

4.3 Notification Management Sequence Diagram

- Using draw.io



Participants involved: Faculty → Notification System → Database → Student → Parent

Steps in the process:

1. **Send Notification** – The Faculty sends a notification through the Notification System.
2. **Store Notification** – The Notification System sends this notification to the Database to be saved.
3. **Success** – The Database confirms back to the Notification System that the notification has been stored successfully.
4. **Receive Notification** – The Notification System sends the notification to the Student.
5. **Receive Notification** – The Notification System also sends the same notification to the Parent.

Description

This diagram shows how a notification is created, saved, and delivered to the right people in the system.

The process starts when the **Faculty** sends a notification, for example, to inform students or parents about something important like attendance, an announcement, or an update. This notification goes to the **Notification System**, which acts as the central point for handling all notifications.

The Notification System first sends this notification to the **Database** to be **stored**, so a record of it is kept. Once the Database saves it successfully, it sends back a "**Success**" confirmation to the Notification System.

After the notification is safely stored, the Notification System delivers it to the people who need to see it — both the **Student** and the **Parent** receive the notification at this stage.

This diagram is important because it shows how the system keeps everyone informed. It makes sure that whenever the Faculty sends any update, it is not only saved for future reference but also properly delivered to both the student and their parent, keeping everyone updated at the same time.

5. UML Class Diagram

The **UML Class Diagram** is a static structural diagram that represents the classes, attributes, methods, and relationships within a system. It provides a blueprint of the system by illustrating how different classes are organized and how they interact with one another. The diagram helps define the overall architecture of the application, showing the responsibilities of each class and the connections between them through relationships such as inheritance, association, aggregation, composition, and dependency.

In the **FaceEntryPro** system, the Class Diagram models the core entities involved in biometric attendance management, including users, authentication, face recognition, attendance, notifications, and administrative functions. It clearly defines the properties and operations of each class while demonstrating how these components collaborate to achieve the system's functionality.

The UML Class Diagram plays an important role during system analysis and design by providing a clear representation of the software structure before implementation. It serves as a reference for developers, simplifies object-oriented design, improves communication among stakeholders, and supports system development, testing, maintenance, and future enhancements.

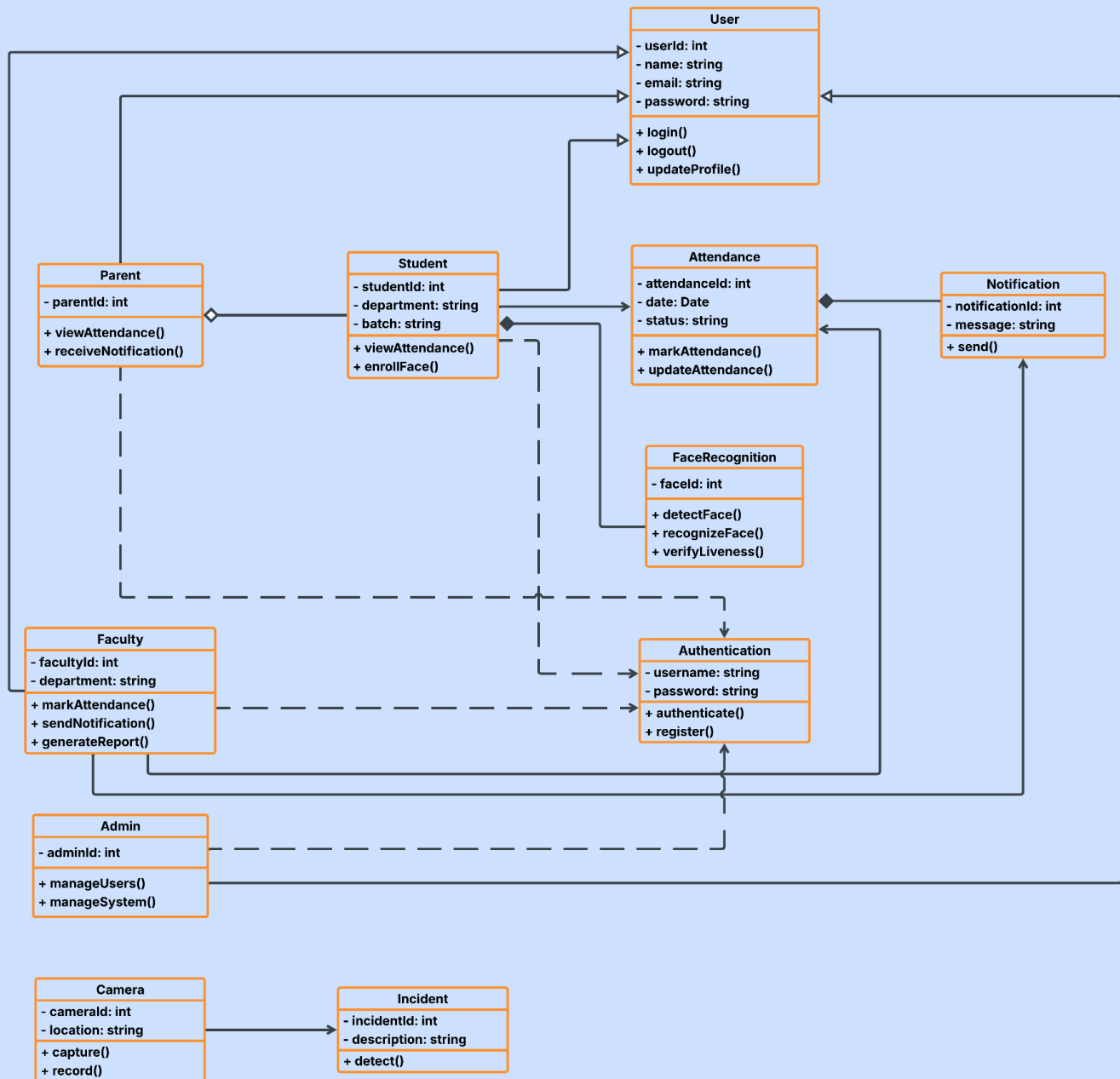
Purpose of the UML Class Diagram - Face Entry Pro

- Represents the static structure of the system.
- Defines the classes, attributes, and methods.
- Illustrates relationships among system components.
- Supports object-oriented system design.
- Provides a blueprint for software implementation.
- Improves maintainability and scalability of the application.

UML Class Diagram - Face Entry Pro

- Using Lucid Chart

Face Entry Pro - UML Diagram



Final Classes

1. User (Super class)
2. Student
3. Faculty
4. Parent
5. Admin
6. Authentication
7. FaceRecognition
8. Attendance
9. Notification
10. Camera
11. Incident

UML Relationships

Inheritance:

1. User <|-- Student
2. User <|-- Faculty
3. User <|-- Parent
4. User <|-- Admin

Association

1. Faculty ----- Attendance
2. Student ----- Attendance
3. Faculty ----- Notification
4. Camera ----- Incident

Aggregation (Open Diamond ◇)

1. Parent ◇----- Student

Composition (Filled Diamond ◆)

1. Attendance ◆----- Notification
2. Student ◇----- Face Recognition

Dependency (Dashed Arrow)

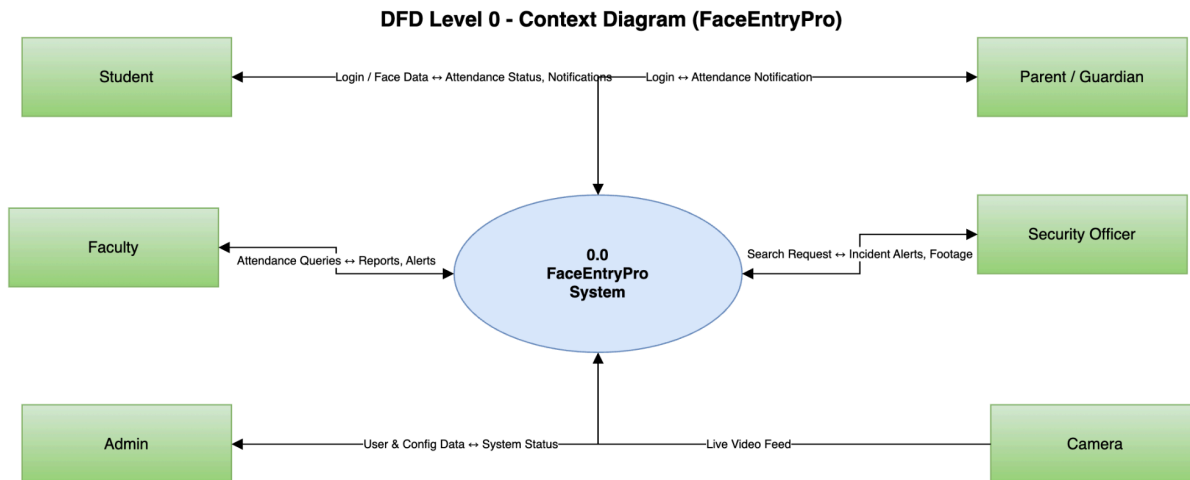
1. Student -----> Authentication
 2. Faculty -----> Authentication
 3. Parent -----> Authentication
 4. Admin -----> Authentication
-

6. Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical tool used to visualize how data moves through a system — where it originates, how it's processed, where it's stored, and where it's ultimately delivered. Unlike flowcharts or activity diagrams, a DFD does not show decision logic or control flow — it focuses purely on the path data takes through the system.

For FaceEntryPro, DFD helps us map out how different stakeholders — Students, Faculty, Admin, Parents/Guardians, Security Officers, and hardware like Cameras — interact with the system, and how information flows through its core functions: biometric attendance, notifications, analytics, and campus security. We built this DFD across three levels of increasing detail, following the standard top-down DFD modeling approach — starting broad, then progressively zooming into specifics.

6.1 Level 0 : Context Diagram



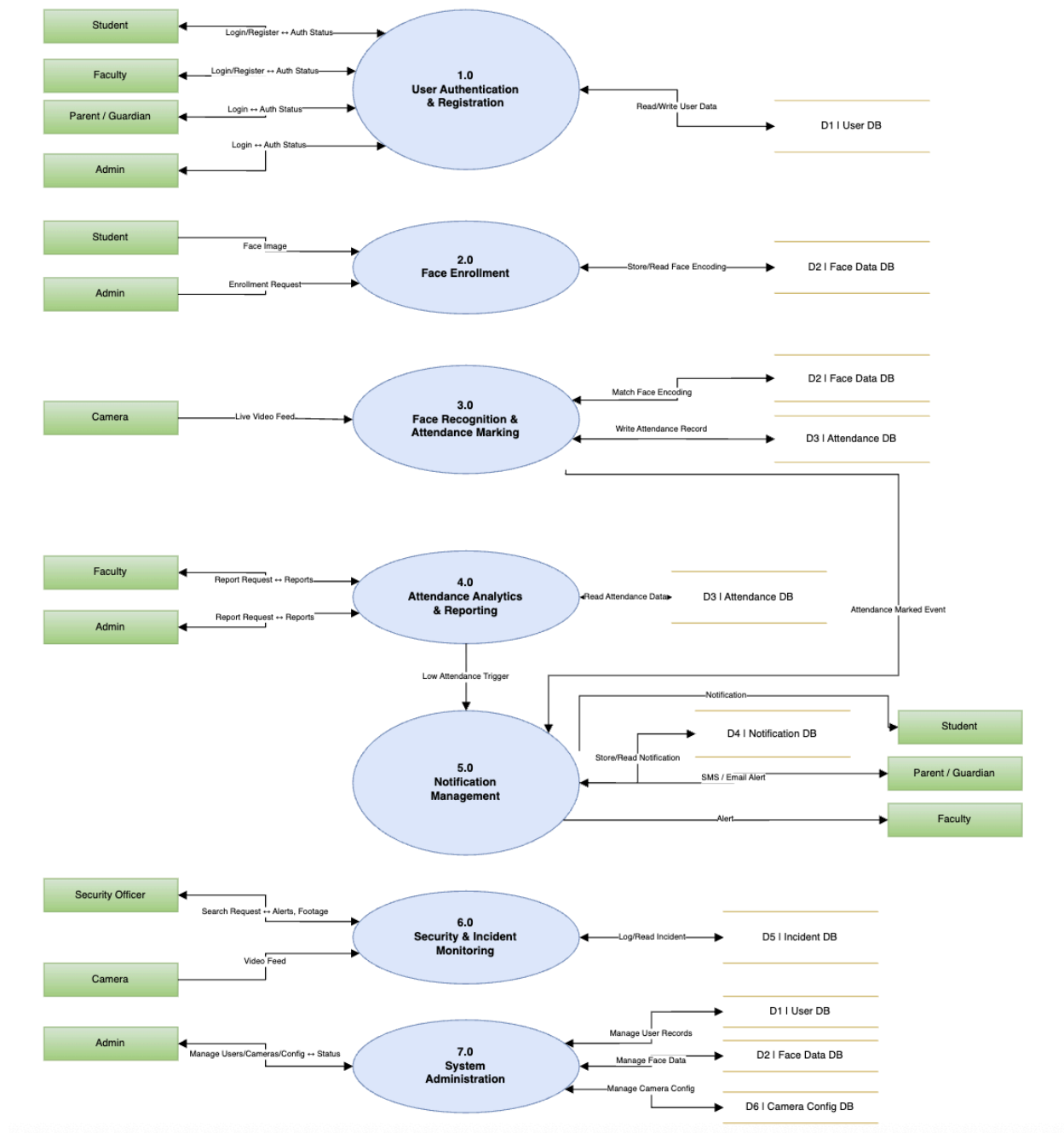
The system as a single black box

At this level, the entire FaceEntryPro system is represented as one unified process — "0.0 FaceEntryPro System". Around it are 6 external entities that interact with the system boundary:

- Student — logs in, submits face data, receives attendance status & notifications
- Faculty — sends queries, receives reports & alerts
- Admin — manages configuration, receives system status
- Parent/Guardian — logs in, receives attendance notifications
- Security Officer — sends search requests, receives incident alerts & footage
- Camera — sends live video feed into the system

This level establishes the system's scope and boundary — showing *who* the system talks to, without revealing what happens inside.

6.2 Level 1 : System Diagram



Breaking the black box into major functional modules

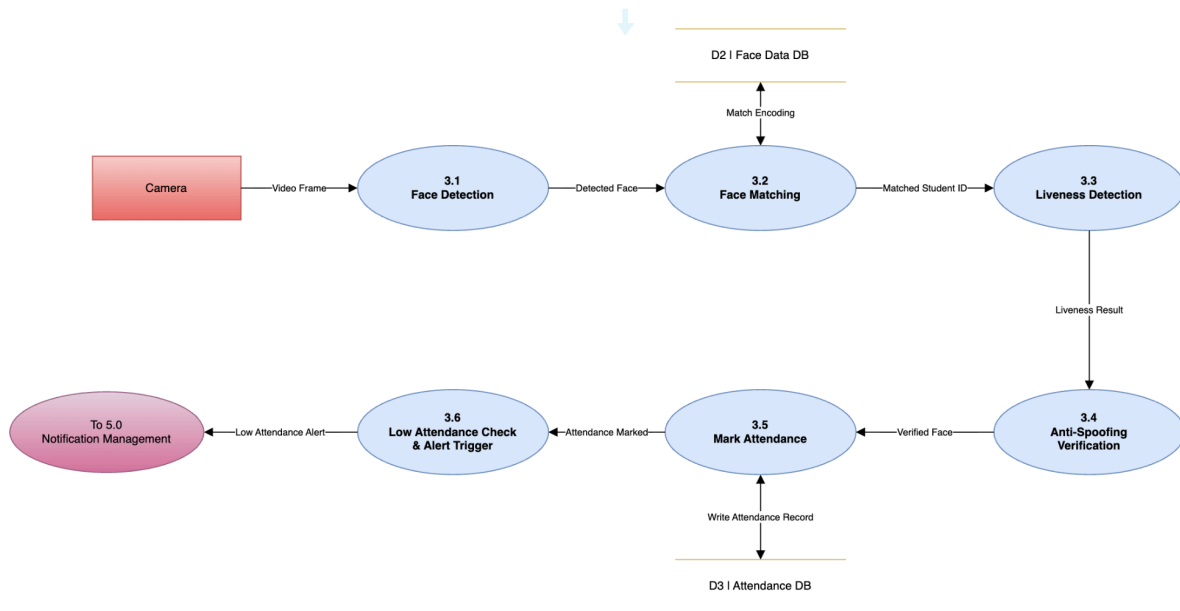
Here, the single Level-0 process is decomposed into 7 major sub-processes, each handling a specific responsibility and connected to its own data store:

Process	Function	Data Store
1.0 User Authentication & Registration	Login/register for all users	D1 – User DB
2.0 Face Enrollment	Capture & store facial data	D2 – Face Data DB
3.0 Face Recognition & Attendance Marking	Detect, match faces, mark attendance	D2, D3 – Attendance DB
4.0 Attendance Analytics & Reporting	Generate reports, detect low attendance	D3
5.0 Notification Management	Send alerts to Student/Parent/Faculty	D4 – Notification DB
6.0 Security & Incident Monitoring	Detect & log incidents, video search	D5 – Incident DB
7.0 System Administration	Manage users, cameras, config, backups	D1, D2, D6

Overall Flow: A user logs in → a student enrolls their face → the Camera feed is matched against stored data to mark attendance → Faculty/Admin generate analytics reports → if attendance is low or gets marked, the Notification module automatically alerts the Student, Parent, and Faculty. Meanwhile, Security Monitoring and System Administration run as parallel supporting modules.

This level reveals the internal architecture — the key building blocks of the system and how data moves between them.

6.3 Level 2 : Process Decomposition



Zooming into the core process

Since Process 3.0 (Face Recognition & Attendance Marking) is the heart of the system, we decomposed it further into 6 detailed sub-processes, showing exactly how a face becomes a marked attendance record:

1. 3.1 Face Detection — locates a face in the camera's video frame
2. 3.2 Face Matching — compares the detected face against stored encodings (D2) to identify the student
3. 3.3 Liveness Detection — confirms the face belongs to a real, live person
4. 3.4 Anti-Spoofing Verification — blocks attempts using photos, videos, or screens
5. 3.5 Mark Attendance — records the verified attendance into the database (D3)
6. 3.6 Low Attendance Check & Alert Trigger — checks the student's attendance trend and triggers a notification if it's low

This level demonstrates how a single high-level process actually functions internally, step-by-step, and is a strong example of how DFDs support progressive refinement.
